

EECE 7205: Fundamentals of Computer
Engineering
Assignment 2

Hamza Ghosheh

September 29, 2026

Contents

1 Problem 1: Bubble Sort	4
1.1 Pseudocode	4
1.2 C++ Implementation	4
1.3 Testing	6
1.4 Question	6
2 Problem 2: Insertion Sort	7
2.1 Pseudocode	7
2.2 C++ Implementation	7
2.3 Testing	9
2.4 Question	10
3 Problem 3: Selection Sort	11
3.1 Pseudocode	11
3.2 C++ Implementation	11
3.3 Testing	13
3.4 Question	13
4 Problem 4: Quick Sort	14
4.1 Pseudocode	14
4.2 C++ Implementation	14
4.3 Testing	16
4.4 Question	16
5 Problem 5: Merge Sort	18
5.1 Pseudocode	18
5.2 C++ Implementation	19
5.3 Testing	22
5.4 Question	24
6 Problem 6: Heap Sort	25
6.1 Pseudocode	25
6.2 C++ Implementation	25
6.3 Max Heap Tree	27
6.4 Testing	28
6.5 Question	29
7 Problem 7: Comparing the Six Algorithms	30
7.1 C++ Implementation	30
7.2 Testing	40
7.3 Comparison Counts Table	42
7.4 Timings Table	42
7.5 First 10 Elements	42
7.6 Question	43

1 Problem 1: Bubble Sort

1.1 Pseudocode

Algorithm 1 Bubble Sort

```
1: function BUBBLESORT(arr, n, comparisonCount, switchCount)
2:    $n \leftarrow \text{SIZEOF}(arr)$ 
3:   for  $iterations \leftarrow 0$  to  $n - 1$  do
4:      $swapTakesPlace \leftarrow \text{false}$ 
5:     for  $i \leftarrow 0$  to  $n - pass - 2$  do
6:       if  $arr[i] > arr[i + 1]$  then
7:         SWAP( $arr[i], arr[i + 1]$ )
8:          $swapTakesPlace \leftarrow \text{true}$ 
9:       end if
10:    end for
11:    if  $swapTakesPlace = \text{false}$  then
12:      break
13:    end if
14:  end for
15:  return  $arr$ 
16: end function
```

1.2 C++ Implementation

```
#include <iostream>
using namespace std;

void printArray(const int arr[], int n){
    for (int i = 0; i < n; i++){
        cout << arr[i] << " ";
    }

    cout << endl;
}

void bubbleSort(int arr[], int n, int& comparisonCount, int
&switchCount){
    int iterations = 0;
    for (int i = 0; i < n - 1; i++){
        bool swapTakesPlace = false;
        for (int j = 0; j < n - 1 - i; j++){
            comparisonCount++;
            if (arr[j] > arr[j + 1]){
                int temp = arr[j];
```

```

        arr[j] = arr[j + 1];
        arr[j + 1] = temp;

        switchCount++;
        swapTakesPlace = true;
    }
}

iterations++;

cout << "Pass " << iterations << ": ";
printArray(arr, n);

if (!swapTakesPlace)
    break;
}
}

int main() {
    int A[] = {5, 7, 14, 19, 23, 32, 34, 62};
    int n = sizeof(A) / sizeof(A[0]);
    int comparisonCount = 0;
    int switches = 0;

    cout << "Original: ";
    printArray(A, n);
    bubbleSort(A, n, comparisonCount, switches);
    cout << "Sorted: ";
    printArray(A, n);

    cout << "Comparisons: " << comparisonCount << endl;
    cout << "Swaps: " << switches << endl;

    return 0;
}

```

1.3 Testing

```
Original: 34 7 23 32 5 62 14 19
Pass 1: 7 23 32 5 34 14 19 62
Pass 2: 7 23 5 32 14 19 34 62
Pass 3: 7 5 23 14 19 32 34 62
Pass 4: 5 7 14 19 23 32 34 62
Pass 5: 5 7 14 19 23 32 34 62
Sorted: 5 7 14 19 23 32 34 62
Comparisons: 25
Swaps: 15
```

Figure 1: Given Array Input

```
Original: 5 7 14 19 23 32 34 62
Pass 1: 5 7 14 19 23 32 34 62
Sorted: 5 7 14 19 23 32 34 62
Comparisons: 7
Swaps: 0
```

Figure 2: Sorted Array Input

1.4 Question

Run your program again on an already sorted copy of A. How many passes and comparisons does it make now? Explain why in your report.

When inputting the already sorted array, bubble sort makes 1 pass and 7 comparisons. The function does not need to perform any swaps during the operation, and hence running through each value once is the best case scenario and the algorithm earlier than expected.

2 Problem 2: Insertion Sort

2.1 Pseudocode

Algorithm 2 Insertion Sort

```
1: function INSERTIONSORT(arr, n, comparisonCount, shiftCount)
2:    $n \leftarrow \text{SIZEOF}(arr)$ 
3:   for  $i \leftarrow 1$  to  $n - 1$  do
4:      $myKey \leftarrow arr[i]$ 
5:      $j \leftarrow i - 1$ 
6:     while  $j \geq 0$  and  $arr[j] > myKey$  do
7:        $A[j + 1] \leftarrow A[j]$ 
8:        $j \leftarrow j - 1$ 
9:     end while
10:     $arr[j + 1] \leftarrow myKey$ 
11:  end for
12:  return  $arr$ 
13: end function
```

2.2 C++ Implementation

```
#include <iostream>
using namespace std;

void printArray(const int arr[], int n){
    for (int i = 0; i < n; i++){
        cout << arr[i] << " ";
    }

    cout << endl;
}

void insertionSort(int arr[], int n, int& comparisonCount,
int &shiftCount){
    for (int i = 1; i < n; i++){ // this loop symbolizes how
        many passes I conducted through the array
        int myKey = arr[i];
        int j = i - 1;

        while (j >= 0 && arr[j] > myKey){
            comparisonCount++;
            arr[j+1] = arr[j];
            shiftCount++;
            j--;
        }
    }
}
```

```

        if (j >= 0){
            comparisonCount++;
        }

        arr[j+1] = myKey;
        cout << "i = " << i << ", key = " << myKey << ": ";
        printArray(arr, n);
    }
}

int main(){
    int A[] = {5, 7, 14, 19, 23, 32, 34, 62};
    int n = sizeof(A) / sizeof(A[0]);
    int comparisonCount = 0;
    int shiftCount = 0;

    cout << "Original: ";
    printArray(A, n);
    cout << endl;
    insertionSort(A, n, comparisonCount, shiftCount);
    cout << endl;
    cout << "Sorted: ";
    printArray(A, n);

    cout << endl;

    cout << "Comparisons: " << comparisonCount << endl;
    cout << "Shifts: " << shiftCount << endl;

    return 0;
}

```

2.3 Testing

```
Original: 34 7 23 32 5 62 14 19

i = 1, key = 7: 7 34 23 32 5 62 14 19
i = 2, key = 23: 7 23 34 32 5 62 14 19
i = 3, key = 32: 7 23 32 34 5 62 14 19
i = 4, key = 5: 5 7 23 32 34 62 14 19
i = 5, key = 62: 5 7 23 32 34 62 14 19
i = 6, key = 14: 5 7 14 23 32 34 62 19
i = 7, key = 19: 5 7 14 19 23 32 34 62

Sorted: 5 7 14 19 23 32 34 62

Comparisons: 20
Shifts: 15
```

Figure 3: Given Array Input

```
Original: 5 7 14 19 23 32 34 62

i = 1, key = 7: 5 7 14 19 23 32 34 62
i = 2, key = 14: 5 7 14 19 23 32 34 62
i = 3, key = 19: 5 7 14 19 23 32 34 62
i = 4, key = 23: 5 7 14 19 23 32 34 62
i = 5, key = 32: 5 7 14 19 23 32 34 62
i = 6, key = 34: 5 7 14 19 23 32 34 62
i = 7, key = 62: 5 7 14 19 23 32 34 62

Sorted: 5 7 14 19 23 32 34 62

Comparisons: 7
Shifts: 0
```

Figure 4: Sorted Array Input

2.4 Question

Which ordering of the same eight numbers produces the fewest shifts, and which produces the most? Run both and explain the result.

The array that produces the fewest shifts is 5,7,14,19,23,32,34,62, which also happens to be an ordered array from least to greatest. Every element is in its correct position, and there is no need to be shifted to the right. In contrast, the array that will produce the most shifts is 62,34,32,23,19,14,7,5, which is ordered from greatest to least. Every new value must be moved past all the other elements to be situated in its correct location.

3 Problem 3: Selection Sort

3.1 Pseudocode

Algorithm 3 Selection Sort

```
1: function SELECTIONSORT(arr, n, comparisonCount, shiftCount)
2:    $n \leftarrow \text{SIZEOF}(arr)$ 
3:   for  $i \leftarrow 0$  to  $n - 2$  do
4:      $minIdx \leftarrow i$ 
5:     for  $j \leftarrow i + 1$  to  $n - 1$  do
6:       if  $arr[j] < arr[minIdx]$  then
7:          $minIdx \leftarrow j$ 
8:       end if
9:     end for
10:    if  $minIdx \neq i$  then
11:      SWAP( $arr[i], arr[minIdx]$ )
12:    end if
13:  end for
14:  return  $arr$ 
15: end function
```

3.2 C++ Implementation

```
void printArray(const int arr[], int n){
    for (int i = 0; i < n; i++){
        cout << arr[i] << " ";
    }

    cout << endl;
}

void selectionSort(int arr[], int n, int& comparisonCount,
int &shiftCount){
    for (int i = 0; i < n - 1; i++){ // this loop symbolizes
        how many passes I conducted through the array
        int minIdx = i;
        for (int j = i + 1; j < n; j++){
            comparisonCount++;
            if (arr[j] < arr[minIdx]){
                minIdx = j;
            }
        }

        cout << "Pass: " << i + 1 << " min = " << arr[minIdx]
            << " at index " << minIdx << " --> ";
    }
}
```

```

        if (minIdx != i){
            int tempVal = arr[i];
            arr[i] = arr[minIdx];
            arr[minIdx] = tempVal;
            shiftCount++;
        }

        printArray(arr, n);
    }
}

int main(){
    int A[] = {34, 7, 23, 32, 5, 62, 14, 19};
    int n = sizeof(A) / sizeof(A[0]);
    int comparisonCount = 0;
    int shiftCount = 0;

    cout << "Original: ";
    printArray(A, n);
    cout << endl;
    selectionSort(A, n, comparisonCount, shiftCount);
    cout << endl;
    cout << "Sorted: ";
    printArray(A, n);

    cout << endl;

    cout << "Comparisons: " << comparisonCount << endl;
    cout << "Swaps: " << shiftCount << endl;

    return 0;
}

```

3.3 Testing

```
Original: 34 7 23 32 5 62 14 19
Pass: 1 min = 5 at index 4 --> 5 7 23 32 34 62 14 19
Pass: 2 min = 7 at index 1 --> 5 7 23 32 34 62 14 19
Pass: 3 min = 14 at index 6 --> 5 7 14 32 34 62 23 19
Pass: 4 min = 19 at index 7 --> 5 7 14 19 34 62 23 32
Pass: 5 min = 23 at index 6 --> 5 7 14 19 23 62 34 32
Pass: 6 min = 32 at index 7 --> 5 7 14 19 23 32 34 62
Pass: 7 min = 34 at index 6 --> 5 7 14 19 23 32 34 62

Sorted: 5 7 14 19 23 32 34 62

Comparisons: 28
Swaps: 5
```

Figure 5: Given Array Input

3.4 Question

Compare the number of swaps with Bubble Sort from Problem 1. Which algorithm makes fewer swaps on A, and why?

For the given array, we see that selection sort has fewer swaps with 5, compared to bubble sort with 15. The reason for this is bubble sort may switch adjacent elements multiple times when iterating throughout the array, whereas selection sort only looks for the smallest value in the entire array and swaps accordingly.

4 Problem 4: Quick Sort

4.1 Pseudocode

Algorithm 4 Quick Sort (Lomuto Partition)

```
1: function QUICKSORT(arr, low, high)
2:   if low < high then
3:     pivotCount  $\leftarrow$  PARTITION(A, low, high)
4:     QUICKSORT(arr, low, pivotCount - 1)
5:     QUICKSORT(arr, pivotCount + 1, high)
6:   end if
7: end function
8: function PARTITION(arr, low, high)
9:   pivot  $\leftarrow$  arr[high]
10:  i  $\leftarrow$  low - 1
11:  for j  $\leftarrow$  low to high - 1 do
12:    if arr[j]  $\leq$  pivot then
13:      i  $\leftarrow$  i + 1
14:      SWAP(arr[i], arr[j])
15:    end if
16:  end for
17:  SWAP(arr[i + 1], arr[high])
18:  return i + 1
19: end function
```

4.2 C++ Implementation

```
#include <iostream>
using namespace std;

void printArray(const int arr[], int n){
    for (int i = 0; i < n; i++){
        cout << arr[i] << " ";
    }

    cout << endl;
}

int partition(int arr[], int n, int& comparisonCount, int
low, int high){
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++){ // this loop
        symbolizes how many passes I conducted through the
        array
```

```

        comparisonCount++;
        if (arr[j] <= pivot){
            i++;
            int num = arr[i];
            arr[i] = arr[j];
            arr[j] = num;
        }
    }
    int num = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = num;

    cout << "pivot = " << arr[i+1] << ": ";
    printArray(arr, n);
    return i + 1;
}

void quickSort(int arr[], int n, int& comparisonCount, int
low, int high){
    if (low < high){
        int pivotCount = partition(arr, n, comparisonCount,
low, high);
        quickSort(arr, n, comparisonCount, low, pivotCount -
1);
        quickSort(arr, n, comparisonCount, pivotCount + 1,
high);
    }
}

int main(){
    int A[] = {5, 7, 14, 19, 23, 32, 34, 62};
    int n = sizeof(A) / sizeof(A[0]);
    int comparisonCount = 0;

    cout << "Original: ";
    printArray(A, n);
    cout << endl;
    quickSort(A, n, comparisonCount, 0, n - 1);
    cout << endl;
    cout << "Comparisons: " << comparisonCount << endl;
    cout << "Sorted: ";
    printArray(A, n);

    return 0;
}

```

4.3 Testing

```
Original: 34 7 23 32 5 62 14 19  
  
pivot = 19: 7 5 14 19 34 62 23 32  
pivot = 14: 7 5 14 19 34 62 23 32  
pivot = 5: 5 7 14 19 34 62 23 32  
pivot = 32: 5 7 14 19 23 32 34 62  
pivot = 62: 5 7 14 19 23 32 34 62  
  
Comparisons: 14  
Sorted: 5 7 14 19 23 32 34 62
```

Figure 6: Given Array Input

```
Original: 5 7 14 19 23 32 34 62  
  
pivot = 62: 5 7 14 19 23 32 34 62  
pivot = 34: 5 7 14 19 23 32 34 62  
pivot = 32: 5 7 14 19 23 32 34 62  
pivot = 23: 5 7 14 19 23 32 34 62  
pivot = 19: 5 7 14 19 23 32 34 62  
pivot = 14: 5 7 14 19 23 32 34 62  
pivot = 7: 5 7 14 19 23 32 34 62  
  
Comparisons: 28  
Sorted: 5 7 14 19 23 32 34 62
```

Figure 7: Sorted Array Input

4.4 Question

Run Quick Sort on an already sorted copy of A and count the comparisons again. Explain why a sorted array is the worst case when the last element is used as the pivot, and suggest one way to avoid it.

When performing quick sort on a sorted array from least to greatest, we see that it makes 28 comparisons. A sorted array is the worst case when the last element is the pivot because the partition is extremely unbalanced. One side contains $n - 1$ elements, and the other side contains 0, and this uneven distribution gives us a worst-case time complexity of $\Theta(n^2)$. One way to avoid this problem is to instead choose a random pivot rather than the last element of an array.

5 Problem 5: Merge Sort

5.1 Pseudocode

Algorithm 5 Merge Sort

```
function PRINTINGSPLITS(arr, low, high)  
  for  $i \leftarrow low$  to  $high$  do  
    PRINT(arr[i])  
  end for  
  PRINT(new line)  
end function
```

```
function MERGESORT(arr, low, high)  
  if  $low < high$  then  
     $mid \leftarrow \lfloor (low + high) / 2 \rfloor$   
    MERGESORT(arr, low, mid)  
    MERGESORT(arr,  $mid + 1$ , high)  
    MERGE(arr, low, mid, high)  
  end if  
end function
```

Algorithm 5 Merge Sort (Continued)

```
function MERGE(arr, low, mid, high)
    x  $\leftarrow$  0
    y  $\leftarrow$  0
    k  $\leftarrow$  low
    leftSize  $\leftarrow$  mid - low + 1
    rightSize  $\leftarrow$  high - mid
    Create leftSplit of size leftSize
    Create rightSplit of size rightSize
    if low < high then
        for i  $\leftarrow$  0 to leftSize - 1 do
            leftSplit[i]  $\leftarrow$  arr[low + i]
        end for
        for j  $\leftarrow$  0 to rightSize - 1 do
            rightSplit[j]  $\leftarrow$  arr[mid + 1 + j]
        end for
        while x < leftSize and y < rightSize do
            if leftSplit[x]  $\leq$  rightSplit[y] then
                arr[k]  $\leftarrow$  leftSplit[x]
                x  $\leftarrow$  x + 1
            else
                arr[k]  $\leftarrow$  rightSplit[y]
                y  $\leftarrow$  y + 1
            end if
            k  $\leftarrow$  k + 1
        end while
        while x < leftSize do
            arr[k]  $\leftarrow$  leftSplit[x]
            x  $\leftarrow$  x + 1
            k  $\leftarrow$  k + 1
        end while
        while y < rightSize do
            arr[k]  $\leftarrow$  rightSplit[y]
            y  $\leftarrow$  y + 1
            k  $\leftarrow$  k + 1
        end while
        PRINTINGSPLITS(arr, low, high)
    end if
end function
```

5.2 C++ Implementation

```
#include <iostream>
using namespace std;
```

```

void printArray(const int arr[], int n){
    for (int i = 0; i < n; i ++){
        cout << arr[i] << " ";
    }

    cout << endl;
}

void printingSplits(const int arr[], int low, int high){
    for (int i = low; i <= high; i++){
        cout << arr[i] << " ";
    }
    cout << endl;
}

void merge(int arr[], int n, int& comparisonCount, int&
mergeTotal, int low, int mid, int high){
    int x = 0;
    int y = 0;
    int k = low;
    int leftSize = mid - low + 1;
    int rightSize = high - mid;
    int* leftSplit = new int[leftSize];
    int* rightSplit = new int[rightSize];

    if (low < high) {
        for (int i = 0; i < leftSize; i++) {
            leftSplit[i] = arr[low + i];
        }

        for (int j = 0; j < rightSize; j++) {
            rightSplit[j] = arr[mid + 1 + j];
        }

        while (x < leftSize && y < rightSize) {
            comparisonCount++;
            if (low == 0 && high == n - 1){
                mergeTotal++;
            }
            if (leftSplit[x] <= rightSplit[y]) {
                arr[k] = leftSplit[x];
                x++;
            }
            else {
                arr[k] = rightSplit[y];
                y++;
            }
            k++;
        }
    }
}

```

```

    }

    while (x < leftSize) {
        arr[k]= leftSplit[x];
        x++;
        k++;
    }

    while (y < rightSize) {
        arr[k] = rightSplit[y];
        y++;
        k++;
    }

    cout << "merged: ";
    printingSplits(arr, low, high);
    delete[] leftSplit;
    delete[] rightSplit;
}

}

void mergeSort(int arr[], int n, int& comparisonCount, int&
mergeTotal, int low, int high){
    if (low < high){
        cout << "split: ";
        printingSplits(arr, low, high);
        int mid = low + (high - low) / 2;
        mergeSort(arr, n, comparisonCount, mergeTotal, low,
            mid);
        mergeSort(arr, n, comparisonCount, mergeTotal, mid +
            1, high);
        merge(arr, n, comparisonCount, mergeTotal, low, mid,
            high);
    }
}

int main(){
    int A[] = {62, 34, 32, 23, 19, 14, 7, 5};
    int n = sizeof(A) / sizeof(A[0]);
    int comparisonCount = 0;
    int mergeTotal = 0;

    cout << "Original: ";
    printArray(A, n);
    cout << endl;
    mergeSort(A, n, comparisonCount, mergeTotal, 0, n - 1);
    cout << endl;
    cout << "Sorted: ";
    printArray(A, n);
    cout << "Comparisons: " << comparisonCount << endl;

```

```
    cout << "(final merge: " << mergeTotal << ")" << endl;  
    return 0;  
}
```

5.3 Testing

```
Original: 34 7 23 32 5 62 14 19  
  
split: 34 7 23 32 5 62 14 19  
split: 34 7 23 32  
split: 34 7  
merged: 7 34  
split: 23 32  
merged: 23 32  
merged: 7 23 32 34  
split: 5 62 14 19  
split: 5 62  
merged: 5 62  
split: 14 19  
merged: 14 19  
merged: 5 14 19 62  
merged: 5 7 14 19 23 32 34 62  
  
Sorted: 5 7 14 19 23 32 34 62  
Comparisons: 17  
(final merge: 7)
```

Figure 8: Given Array Input

```

Original: 5 7 14 19 23 32 34 62

split: 5 7 14 19 23 32 34 62
split: 5 7 14 19
split: 5 7
merged: 5 7
split: 14 19
merged: 14 19
merged: 5 7 14 19
split: 23 32 34 62
split: 23 32
merged: 23 32
split: 34 62
merged: 34 62
merged: 23 32 34 62
merged: 5 7 14 19 23 32 34 62

Sorted: 5 7 14 19 23 32 34 62
Comparisons: 12
(final merge: 4)

```

Figure 9: Sorted Array Input (Least to Greatest)

```

Original: 62 34 32 23 19 14 7 5

split: 62 34 32 23 19 14 7 5
split: 62 34 32 23
split: 62 34
merged: 34 62
split: 32 23
merged: 23 32
merged: 23 32 34 62
split: 19 14 7 5
split: 19 14
merged: 14 19
split: 7 5
merged: 5 7
merged: 5 7 14 19
merged: 5 7 14 19 23 32 34 62

Sorted: 5 7 14 19 23 32 34 62
Comparisons: 12
(final merge: 4)

```

Figure 10: Sorted Array Input (Greatest to Least)

5.4 Question

Run Merge Sort on sorted and reversed copies of A. Explain why the number of comparisons changes much less than Bubble Sort or Quick Sort, and state how much extra memory Merge Sort requires.

The number of comparisons changes much less since it follows the same method of divide-and-conquer regardless of the order of the array. Merge sort continues to split arrays in halves and re-merges them together in the proper sorting method. The additional memory Merge Sort requires is $\Theta(n)$ due to the fact there are temporary arrays being created.

6 Problem 6: Heap Sort

6.1 Pseudocode

Algorithm 6 Heap Sort

```
1: function HEAPSORT(arr, n, swapCount)
2:   n  $\leftarrow$  SIZEOF(arr)
3:   for i  $\leftarrow$   $\lfloor n/2 \rfloor - 1$  downto 0 do
4:     SIFTDOWN(arr, i, n)
5:   end for
6:   for end  $\leftarrow$  n - 1 downto 1 do
7:     SWAP(arr[0], arr[end])
8:     SIFTDOWN(arr, 0, end)
9:   end for
10:  Display array, heap and all sorted values accordingly
11:  return arr
12: end function

13: function SIFTDOWN(arr, root, size, swapCount)
14:   while  $2 \cdot \text{root} + 1 < \text{size}$  do
15:     childNode  $\leftarrow$   $2 \cdot \text{root} + 1$ 
16:     if childNode + 1 < size and arr[childNode + 1] > arr[childNode]
17:       then
18:         childNode  $\leftarrow$  childNode + 1
19:       end if
20:       if arr[root]  $\geq$  arr[childNode] then
21:         break
22:       end if
23:       SWAP(arr[root], arr[childNode])
24:       root  $\leftarrow$  childNode
25:     end while
26: end function
```

6.2 C++ Implementation

```
#include <iostream>
using namespace std;

void printArray(const int arr[], int n){
    for (int i = 0; i < n; i++){
        cout << arr[i] << " ";
    }
    cout << endl;
}
```

```

void siftDown(int arr[], int root, int size, int& swapCount)
{
    while (2 * root + 1 < size){
        int childNode = 2 * root + 1;

        if (childNode + 1 < size && arr[childNode + 1] > arr
            [childNode]){
            childNode = childNode + 1;
        }

        if (arr[root] >= arr[childNode]){
            break;
        }

        int tempVal = arr[root];
        arr[root] = arr[childNode];
        arr[childNode] = tempVal;
        swapCount++;
        root = childNode;
    }
}

void heapSort(int arr[], int n, int& swapCount){
    for (int i = n / 2 - 1; i >= 0; i--){
        siftDown(arr, i, n, swapCount);
    }

    cout << "Heap: ";
    printArray(arr, n);

    for (int lastVal = n - 1; lastVal >= 1; lastVal--){
        int tempVal = arr[0];
        arr[0] = arr[lastVal];
        arr[lastVal] = tempVal;

        siftDown(arr, 0, lastVal, swapCount);

        cout << "end= " << lastVal << ": heap: ";
        printArray(arr, lastVal);
        cout << "        sorted: ";
        printArray(arr + lastVal, n - lastVal);
        cout << endl;
    }
}

int main(){
    int A[] = {4, 7, 23, 32, 5, 62, 14, 19};
    int n = sizeof(A) / sizeof(A[0]);

```

```

    int swapCount = 0;

    cout << "Original: ";

    printArray(A, n);
    cout << endl;
    heapSort(A, n, swapCount);
    cout << endl;
    cout << "Sorted: ";
    printArray(A, n);
    cout << endl;
    cout << "Swaps in siftDown: " << swapCount;

    cout << endl;

    return 0;
}

```

6.3 Max Heap Tree

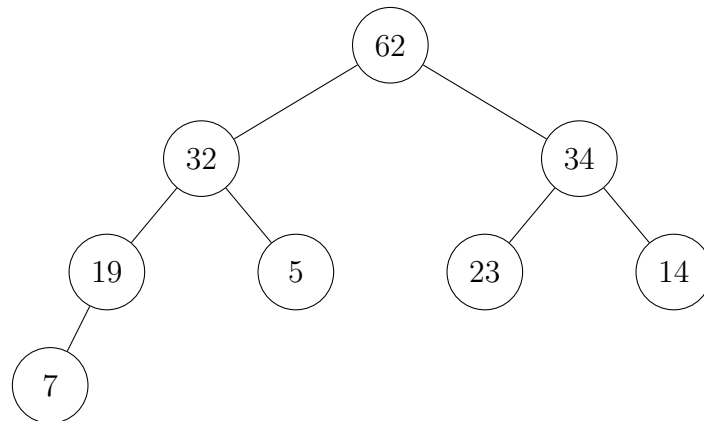


Figure 11: Max Heap Constructed from the Given Array

6.4 Testing

```
Original: 4 7 23 32 5 62 14 19
Heap: 62 32 23 19 5 4 14 7
end = 7: heap: 32 19 23 7 5 4 14
        sorted: 62

end = 6: heap: 23 19 14 7 5 4
        sorted: 32 62

end = 5: heap: 19 7 14 4 5
        sorted: 23 32 62

end = 4: heap: 14 7 5 4
        sorted: 19 23 32 62

end = 3: heap: 7 4 5
        sorted: 14 19 23 32 62

end = 2: heap: 5 4
        sorted: 7 14 19 23 32 62

end = 1: heap: 4
        sorted: 5 7 14 19 23 32 62

Sorted: 4 5 7 14 19 23 32 62

Swaps in siftDown: 12
```

Figure 12: Sorted Array Input (Greatest to Least)

6.5 Question

Is Heap Sort stable? Explain using a small example containing two equal values.

Heap Sort is not stable, since a stable sorting algorithm preserves a relative order of what the original array looks like. What is unique about heap sort is that it performs swaps between elements that can be distant from one another in the array, meaning that equal elements can change their relative order. As an example, consider two equal values labeled to distinguish them:

$$[5_x, 5_y, 1]$$

Even though 5_x visually and technically appears before 5_y , the heap operations can cause 5_y to appear before 5_x . Thus, equal elements do not necessarily retain their original order when being heap sort is being run.

7 Problem 7: Comparing the Six Algorithms

7.1 C++ Implementation

```
#include <iostream>
#include <chrono>
#include <cstdlib>
using namespace std;

void printArray(const int arr[], int n){
    for (int i = 0; i < n; i++){
        cout << arr[i] << " ";
    }

    cout << endl;
}

void bubbleSort(int arr[], int n, int& comparisonCount, int
&switchCount){
    int iterations = 0;
    for (int i = 0; i < n - 1; i++){
        bool swapTakesPlace = false;
        for (int j = 0; j < n - 1 - i; j++){
            comparisonCount++;
            if (arr[j] > arr[j + 1]){
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;

                switchCount++;
                swapTakesPlace = true;
            }
        }
        iterations++;
        if (!swapTakesPlace)
            break;
    }
}

void insertionSort(int arr[], int n, int& comparisonCount,
int &shiftCount){
    for (int i = 1; i < n; i++){
        int myKey = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] > myKey){
            comparisonCount++;
            arr[j+1] = arr[j];
            shiftCount++;
        }
    }
}
```

```

        j--;
    }
    if (j >= 0){
        comparisonCount++;
    }
    arr[j+1] = myKey;
}
}

void selectionSort(int arr[], int n, int& comparisonCount,
int &shiftCount){
    for (int i = 0; i < n - 1; i++){
        int minIdx = i;
        for (int j = i + 1; j < n; j++){
            comparisonCount++;
            if (arr[j] < arr[minIdx]){
                minIdx = j;
            }
        }

        if (minIdx != i){
            int tempVal = arr[i];
            arr[i] = arr[minIdx];
            arr[minIdx] = tempVal;
            shiftCount++;
        }
    }
}

int partition(int arr[], int n, int& comparisonCount, int
low, int high){
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++){
        comparisonCount++;
        if (arr[j] <= pivot){
            i++;
            int num = arr[i];
            arr[i] = arr[j];
            arr[j] = num;
        }
    }
    int num = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = num;

    return i + 1;
}

```

```

void quickSort(int arr[], int n, int& comparisonCount, int
low, int high){
    if (low < high){
        int pivotCount = partition(arr, n, comparisonCount,
low, high);
        quickSort(arr, n, comparisonCount, low, pivotCount -
1);
        quickSort(arr, n, comparisonCount, pivotCount + 1,
high);
    }
}

void merge(int arr[], int n, int& comparisonCount, int&
mergeTotal, int low, int mid, int high){
    int x = 0;
    int y = 0;
    int k = low;
    int leftSize = mid - low + 1;
    int rightSize = high - mid;
    int* leftSplit = new int[leftSize];
    int* rightSplit = new int[rightSize];

    if (low < high) {
        for (int i = 0; i < leftSize; i++) {
            leftSplit[i] = arr[low + i];
        }

        for (int j = 0; j < rightSize; j++) {
            rightSplit[j] = arr[mid + 1 + j];
        }

        while (x < leftSize && y < rightSize) {
            comparisonCount++;
            if (low == 0 && high == n - 1){
                mergeTotal++;
            }
            if (leftSplit[x] <= rightSplit[y]) {
                arr[k] = leftSplit[x];
                x++;
            }
            else {
                arr[k] = rightSplit[y];
                y++;
            }
            k++;
        }

        while (x < leftSize) {
            arr[k] = leftSplit[x];
            x++;
        }
    }
}

```

```

        k++;
    }

    while (y < rightSize) {
        arr[k] = rightSplit[y];
        y++;
        k++;
    }

    delete[] leftSplit;
    delete[] rightSplit;
}

void mergeSort(int arr[], int n, int& comparisonCount, int&
mergeTotal, int low, int high){
    if (low < high){
        int mid = low + (high - low) / 2;
        mergeSort(arr, n, comparisonCount, mergeTotal, low,
mid);
        mergeSort(arr, n, comparisonCount, mergeTotal, mid +
1, high);
        merge(arr, n, comparisonCount, mergeTotal, low, mid,
high);
    }
}

void siftDown(int arr[], int root, int size, int&
comparisonCount){
    while (2 * root + 1 < size){
        int childNode = 2 * root + 1;
        if (childNode + 1 < size){
            comparisonCount++;
            if (arr[childNode + 1] > arr[childNode]){
                childNode = childNode + 1;
            }
        }

        comparisonCount++;

        if (arr[root] >= arr[childNode]){
            break;
        }

        int tempVal = arr[root];
        arr[root] = arr[childNode];
        arr[childNode] = tempVal;
        root = childNode;
    }
}

```

```

void heapSort(int arr[], int n, int& comparisonCount){
    for (int i = n / 2 - 1; i >= 0; i--){
        siftDown(arr, i, n, comparisonCount);
    }

    for (int lastVal = n - 1; lastVal >= 1; lastVal--){
        int tempVal = arr[0];
        arr[0] = arr[lastVal];
        arr[lastVal] = tempVal;
        siftDown(arr, 0, lastVal, comparisonCount);
    }
}

int main(){

    int bubble = 0;
    int switches = 0;
    int insertion = 0;
    int shiftCount = 0;
    int selectionCount = 0;
    int selection = 0;
    int quick = 0;
    int merge = 0;
    int mergeTotal = 0;
    int heap = 0;

    int originalArray[] = {34, 7, 23, 32, 5, 62, 14, 19};
    int sortedArray[] = {5, 7, 14, 19, 23, 32, 34, 62};
    int reversedArray[] = {62, 34, 32, 23, 19, 14, 7, 5};

    int originalVal = sizeof(originalArray) / sizeof(
        originalArray[0]);
    int sortedVal = sizeof(sortedArray) / sizeof(sortedArray
        [0]);
    int reversedVal = sizeof(reversedArray) / sizeof(
        reversedArray[0]);

    int* bubbleArray = new int[originalVal];
    int* insertionArray = new int[originalVal];
    int* selectionArray = new int[originalVal];
    int* quickArray = new int[originalVal];
    int* mergeArray = new int[originalVal];
    int* heapArray = new int[originalVal];

    for (int i = 0; i < originalVal; i++){
        bubbleArray[i] = originalArray[i];
        insertionArray[i] = originalArray[i];
        selectionArray[i] = originalArray[i];
        quickArray[i] = originalArray[i];
    }
}

```

```

        mergeArray[i] = originalArray[i];
        heapArray[i] = originalArray[i];
    }

    bubbleSort(bubbleArray, originalVal, bubble, switches);
    insertionSort(insertionArray, originalVal, insertion,
        shiftCount);
    selectionSort(selectionArray, originalVal, selection,
        selectionCount);
    quickSort(quickArray, originalVal, quick, 0, originalVal
        - 1);
    mergeSort(mergeArray, originalVal, merge, mergeTotal, 0,
        originalVal - 1);
    heapSort(heapArray, originalVal, heap);
    cout << endl;

    cout << endl;
    cout << "Original Array Comparison Counts:" << endl;
    cout << endl;
    cout << "Bubble Sort: " << bubble << endl;
    cout << "Insertion Sort: " << insertion << endl;
    cout << "Selection Sort: " << selection << endl;
    cout << "Quick Sort: " << quick << endl;
    cout << "Merge Sort: " << merge << endl;
    cout << "Heap Sort: " << heap << endl;

    bubble = 0;
    switches = 0;
    insertion = 0;
    shiftCount = 0;
    selectionCount = 0;
    selection = 0;
    quick = 0;
    merge = 0;
    mergeTotal = 0;
    heap = 0;

    for (int i = 0; i < sortedVal; i++){
        bubbleArray[i] = sortedArray[i];
        insertionArray[i] = sortedArray[i];
        selectionArray[i] = sortedArray[i];
        quickArray[i] = sortedArray[i];
        mergeArray[i] = sortedArray[i];
        heapArray[i] = sortedArray[i];
    }

    bubbleSort(bubbleArray, sortedVal, bubble, switches);
    insertionSort(insertionArray, sortedVal, insertion,
        shiftCount);

```

```

selectionSort(selectionArray, sortedVal, selection,
               selectionCount);
quickSort(quickArray, sortedVal, quick, 0, sortedVal -
          1);
mergeSort(mergeArray, sortedVal, merge, mergeTotal, 0,
          sortedVal - 1);
heapSort(heapArray, sortedVal, heap);

cout << endl;
cout << "Sorted Array Comparison Counts:" << endl;
cout << "Bubble Sort: " << bubble << endl;
cout << "Insertion Sort: " << insertion << endl;
cout << "Selection Sort: " << selection << endl;
cout << "Quick Sort: " << quick << endl;
cout << "Merge Sort: " << merge << endl;
cout << "Heap Sort: " << heap << endl;

bubble = 0;
switches = 0;
insertion = 0;
shiftCount = 0;
selection = 0;
quick = 0;
merge = 0;
mergeTotal = 0;
heap = 0;

for (int i = 0; i < reversedVal; i++){
    bubbleArray[i] = reversedArray[i];
    insertionArray[i] = reversedArray[i];
    selectionArray[i] = reversedArray[i];
    quickArray[i] = reversedArray[i];
    mergeArray[i] = reversedArray[i];
    heapArray[i] = reversedArray[i];
}

bubbleSort(bubbleArray, reversedVal, bubble, switches);
insertionSort(insertionArray, reversedVal, insertion,
               shiftCount);
selectionSort(selectionArray, reversedVal, selection,
               selectionCount);
quickSort(quickArray, reversedVal, quick, 0, reversedVal
          - 1);
mergeSort(mergeArray, reversedVal, merge, mergeTotal, 0,
          reversedVal - 1);
heapSort(heapArray, reversedVal, heap);

cout << endl;
cout << "Reversed Array Comparison Counts:" << endl;
cout << endl;

```

```

cout << "Bubble Sort: " << bubble << endl;
cout << "Insertion Sort: " << insertion << endl;
cout << "Selection Sort: " << selection << endl;
cout << "Quick Sort: " << quick << endl;
cout << "Merge Sort: " << merge << endl;
cout << "Heap Sort: " << heap << endl;

delete[] bubbleArray;
delete[] insertionArray;
delete[] selectionArray;
delete[] quickArray;
delete[] mergeArray;
delete[] heapArray;

cout << endl;
cout << "Sorting Times:" << endl;

int sizes[] = {1000, 5000, 10000};
int sizeTotal = sizeof(sizes) / sizeof(sizes[0]);

for (int a = 0; a < sizeTotal; a++){
    int b = sizes[a];
    cout << "\nArray Size: " << b << endl;
    int* newArr = new int[b];
    for (int i = 0; i < b; i++){
        newArr[i] = rand() % 100000 + 1;
    }

    int* rand_bubble = new int[b];
    int* rand_insert = new int[b];
    int* rand_select = new int[b];
    int* rand_quick = new int[b];
    int* rand_merge = new int[b];
    int* rand_heap = new int[b];

    for (int i = 0; i < b; i++){
        rand_bubble[i] = newArr[i];
        rand_insert[i] = newArr[i];
        rand_select[i] = newArr[i];
        rand_quick[i] = newArr[i];
        rand_merge[i] = newArr[i];
        rand_heap[i] = newArr[i];
    }

    bubble = 0;
    switches = 0;
    auto beginBubble = chrono::high_resolution_clock::
        now();
    bubbleSort(rand_bubble, b, bubble, switches);

```

```

auto endBubble = chrono::high_resolution_clock::now
();
auto bubbleTime = chrono::duration_cast<chrono::
    microseconds>(endBubble - beginBubble).count();
cout << "Bubble Sort: " << bubbleTime << "
    microseconds" << endl;

insertion = 0;
shiftCount = 0;
auto beginInsertion = chrono::high_resolution_clock
::now();
insertionSort(rand_insert, b, insertion, shiftCount)
;
auto endInsertion = chrono::high_resolution_clock::
    now();
auto insertClock = chrono::duration_cast<chrono::
    microseconds>(endInsertion - beginInsertion).
    count();
cout << "Insertion Sort: " << insertClock << "
    microseconds" << endl;

selection = 0;
int selectionCount = 0;
auto beginSelection = chrono::high_resolution_clock
::now();
selectionSort(rand_select, b, selection,
    selectionCount);
auto endSelection = chrono::high_resolution_clock::
    now();
auto selectClock = chrono::duration_cast<chrono::
    microseconds>(endSelection - beginSelection).
    count();
cout << "Selection Sort: " << selectClock << "
    microseconds" << endl;

quick = 0;
auto beginQuick = chrono::high_resolution_clock::now
();
quickSort(rand_quick, b, quick, 0, b - 1);
auto endQuick = chrono::high_resolution_clock::now()
;
auto quickClock = chrono::duration_cast<chrono::
    microseconds>(endQuick - beginQuick).count();
cout << "Quick Sort: " << quickClock << "
    microseconds" << endl;

merge = 0;
mergeTotal = 0;
auto beginMerge = chrono::high_resolution_clock::now
();

```

```

mergeSort(rand_merge, b, merge, mergeTotal, 0, b -
1);
auto endMerge = chrono::high_resolution_clock::now()
;
auto mergeClock = chrono::duration_cast<chrono::
microseconds>(endMerge - beginMerge).count();
cout << "Merge Sort: " << mergeClock << "
microseconds" << endl;

heap = 0;
auto beginHeap = chrono::high_resolution_clock::now
();
heapSort(rand_heap, b, heap);
auto endHeap = chrono::high_resolution_clock::now();
auto heapClock = chrono::duration_cast<chrono::
microseconds>(endHeap - beginHeap).count();
cout << "Heap Sort: " << heapClock << " microseconds"
<< endl;

cout << endl;
cout << "First 10 Sorted Elements:" << endl;
cout << "Bubble Sort: ";
printArray(rand_bubble, 10);
cout << "Insertion Sort: ";
printArray(rand_insert, 10);
cout << "Selection Sort: ";
printArray(rand_select, 10);
cout << "Quick Sort: ";
printArray(rand_quick, 10);
cout << "Merge Sort: ";
printArray(rand_merge, 10);
cout << "Heap Sort: ";
printArray(rand_heap, 10);

delete[] newArr;
delete[] rand_bubble;
delete[] rand_insert;
delete[] rand_select;
delete[] rand_quick;
delete[] rand_merge;
delete[] rand_heap;
}

return 0;
}

```

7.2 Testing

Original Array Comparison Counts:

Bubble Sort: 25
Insertion Sort: 20
Selection Sort: 28
Quick Sort: 14
Merge Sort: 17
Heap Sort: 26

Sorted Array Comparison Counts:

Bubble Sort: 7
Insertion Sort: 7
Selection Sort: 28
Quick Sort: 28
Merge Sort: 12
Heap Sort: 27

Reversed Array Comparison Counts:

Bubble Sort: 28
Insertion Sort: 28
Selection Sort: 28
Quick Sort: 28
Merge Sort: 12
Heap Sort: 24

Figure 13: Sorted Array Input (Greatest to Least)

Sorting Times:

Array Size: 1000

Bubble Sort: 4031 microseconds

Insertion Sort: 542 microseconds

Selection Sort: 2057 microseconds

Quick Sort: 63 microseconds

Merge Sort: 182 microseconds

Heap Sort: 88 microseconds

Array Size: 5000

Bubble Sort: 66430 microseconds

Insertion Sort: 15210 microseconds

Selection Sort: 27181 microseconds

Quick Sort: 360 microseconds

Merge Sort: 817 microseconds

Heap Sort: 497 microseconds

Array Size: 10000

Bubble Sort: 172536 microseconds

Insertion Sort: 54108 microseconds

Selection Sort: 104573 microseconds

Quick Sort: 757 microseconds

Merge Sort: 1593 microseconds

Heap Sort: 1058 microseconds

Figure 14: Sorted Array Input (Greatest to Least)

7.3 Comparison Counts Table

Table 1: Comparison Counts for the Six Sorting Algorithms

Algorithm	Original Array	Sorted Array	Reversed Array
Bubble Sort	25	7	28
Insertion Sort	20	7	28
Selection Sort	28	28	28
Quick Sort	14	28	28
Merge Sort	17	12	12
Heap Sort	26	27	24

7.4 Timings Table

Table 2: Sorting Times for Random Arrays

Algorithm	1,000	5,000	10,000
Bubble Sort	4031	66430	172536
Insertion Sort	542	15210	54108
Selection Sort	2057	27181	104573
Quick Sort	63	360	757
Merge Sort	182	817	1593
Heap Sort	88	497	1058

Table 3: *

Execution times are measured in microseconds (μs).

7.5 First 10 Elements

```
First 10 Sorted Elements:
Bubble Sort: 111 237 261 477 520 523 561 563 626 680
Insertion Sort: 111 237 261 477 520 523 561 563 626 680
Selection Sort: 111 237 261 477 520 523 561 563 626 680
Quick Sort: 111 237 261 477 520 523 561 563 626 680
Merge Sort: 111 237 261 477 520 523 561 563 626 680
Heap Sort: 111 237 261 477 520 523 561 563 626 680
```

Figure 15: First 10 Elements (Array Size 1000)

```

First 10 Sorted Elements:
Bubble Sort: 11 15 26 27 28 38 51 60 119 161
Insertion Sort: 11 15 26 27 28 38 51 60 119 161
Selection Sort: 11 15 26 27 28 38 51 60 119 161
Quick Sort: 11 15 26 27 28 38 51 60 119 161
Merge Sort: 11 15 26 27 28 38 51 60 119 161
Heap Sort: 11 15 26 27 28 38 51 60 119 161

```

Figure 16: First 10 Elements (Array Size 5000)

```

First 10 Sorted Elements:
Bubble Sort: 7 10 17 27 34 42 43 48 71 83
Insertion Sort: 7 10 17 27 34 42 43 48 71 83
Selection Sort: 7 10 17 27 34 42 43 48 71 83
Quick Sort: 7 10 17 27 34 42 43 48 71 83
Merge Sort: 7 10 17 27 34 42 43 48 71 83
Heap Sort: 7 10 17 27 34 42 43 48 71 83

```

Figure 17: First 10 Elements (Array Size 10000)

7.6 Question

Which algorithms grow like n^2 and which grow like $n \log n$? Do your measurements agree with theory? Which would you choose for a large random array and which for an almost-sorted array? Explain.

The worst case time complexities with $\Theta(n^2)$ are Bubble Sort, Insertion Sort, and Selection Sort. In contrast, looking at the time complexity $\Theta(n \log n)$, we see that it is found Merge Sort and Heap Sort. Even though Quick Sort has an average-case complexity of $\Theta(n \log n)$, it can sometimes become to $\Theta(n^2)$ in the worst case. My measurements agree with these theoretical results, due to the fact that a random-array size increased from 1,000 to 10,000 elements causes Bubble Sort, Insertion Sort, and Selection Sort to increase much more rapidly than Quick Sort, Merge Sort, and Heap Sort. The most ideal sorting method for a large random array would be quick sort since it had the lowest execution time. Similarly, Insertion Sort would be a good choice for an almost-sorted array since not that many elements need to be shifted into the correct location when most elements are already close to their correct positions.

8 References

For references, I used the slides on Canvas to help understand the pseudocode better as well as implementing the code into C++. All references and external

resources used in writing the C++ implementation of each problem are cited directly within the original C++ source code. Please see the submitted C++ file for the corresponding links and citations on GitHub for your reference.

Problem 1

- Line 23: <https://chatgpt.com/c/6abc52b0-e02c-83e9-be7d-60d9bc686e83>

Problem 4

- Line 23: <https://chatgpt.com/c/6abc54de-50dc-83e9-b39d-a2110ac9313a>

Problem 5

- Line 23: <https://chatgpt.com/c/6abc4eee-4b24-83e9-8add-4f93ebd30c1e>

Problem 6

- Line 19: <https://chatgpt.com/c/6abc424c-dd5c-83e9-99cd-d79c68292f15>

Problem 7

- Line 426: https://en.cppreference.com/cpp/chrono/high_resolution_clock
- Line 434: <https://www.geeksforgeeks.org/cpp/chrono-in-c/>